

Adaptive Semantic-Velocity Chunking with Dynamic-Relevant RAG (DR-RAG)

A Major Project Report

Submitted in Partial Fulfillment of the Requirements
for the Degree of Bachelor of Technology

Rajeev Gupta

Department of Computer Science and Engineering

May 8, 2026

Contents

1	Motivation	3
2	Literature Review	4
2.1	Chunking Strategies in RAG Systems	4
2.1.1	Fixed-Size and Sentence-Based Chunking	4
2.1.2	Recursive and Semantic Chunking	4
2.1.3	Proposition-Based Chunking	6
2.1.4	Page-Level and Hierarchical (RAPTOR) Chunking	6
2.2	Retrieval Architectures	6
2.2.1	Single-Stage Dense Retrieval	6
2.2.2	Multi-Stage and Iterative Retrieval	6
2.2.3	Graph-Based Retrieval	6
2.3	Research Gaps	7
3	Methodology	8
3.1	Objective	8
3.2	Phase I: Context-Augmented Agentic Chunking (CAAC)	8
3.2.1	Overview	8
3.2.2	Two-Pass Document Segmentation	8
3.2.3	Boundary Detector Training Data	9
3.2.4	Semantic Velocity	9
3.2.5	Context-Augmented Chunk Construction	10
3.2.6	Token Budget Scaling	10
3.2.7	Context Retention Score	10
3.2.8	Embedding and Indexing	10
3.3	Phase II: Dynamic-Relevant RAG (DR-RAG)	10
3.3.1	Overview	10
3.3.2	Step 1 — Query Encoding and Routing	11
3.3.3	Step 2 — Anchor Retrieval	11
3.3.4	Step 3 — Evidence Mining (Multi-Hop Only)	11
3.3.5	Context Assembly and Response Generation	12
3.4	Ablation Study	12

Abstract

Retrieval-Augmented Generation (RAG) systems face fundamental limitations when processing long, structured documents such as legal contracts and academic papers. Standard fixed-size chunking disrupts semantic coherence, while single-stage retrieval fails to support multi-hop reasoning across weakly linked evidence.

This work proposes **Adaptive Semantic-Velocity Chunking with Dynamic-Relevant RAG (DR-RAG)**, a two-phase framework that addresses both challenges. Phase I introduces **Context-Augmented Agentic Chunking (CAAC)**, which replaces static chunking with Small Language Model (SLM)-guided semantic boundaries. Each chunk is enriched with a rolling summary of preceding context, ensuring preservation of both local meaning and global document flow. Semantic velocity, defined as embedding drift between consecutive chunks, dynamically adjusts chunk size and summary length.

Phase II introduces a **classifier-routed two-stage retrieval mechanism**. The first stage retrieves anchor documents using cosine similarity. For complex queries, a second stage generates expanded queries from intermediate facts to retrieve additional supporting evidence. The final response is generated using the union of anchor and evidence sets.

The proposed framework is designed to improve recall, contextual coherence, multi-hop reasoning accuracy, and hallucination resistance compared to conventional RAG systems. Experimental evaluation is planned using HotpotQA [1] and CUAD [2] benchmarks.

1 Motivation

Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding large language model (LLM) responses in external, verifiable knowledge. By retrieving relevant document passages at inference time, RAG systems reduce hallucination and extend the effective knowledge horizon of LLMs beyond their training cut-off. Despite this promise, current RAG pipelines exhibit critical structural weaknesses when applied to long, domain-specific documents such as legal contracts, academic research papers, and technical standards. Three fundamental limitations motivate the design of the proposed DR-RAG framework.

Limitation 1: Semantic Fragmentation due to Fixed-Size Chunking. The overwhelming majority of deployed RAG systems partition source documents into fixed-size token windows of 256–512 tokens with optional overlap. This strategy is computationally convenient but semantically blind. In legal documents, a single obligation clause may span several hundred words across multiple sentences; splitting it at a token boundary severs the conditional relationship between its premise and its consequence. Similarly, in academic papers, a mathematical proposition and its proof are logically inseparable, yet fixed-size chunking routinely places them in different chunks. The retriever then returns a fragment that contains only part of the relevant reasoning, causing the generator to produce incomplete or factually incorrect answers. Overlap-based mitigation inflates the vector index size without resolving the underlying boundary problem.

Limitation 2: Context Decay in Long Documents. Even when individual chunks are semantically coherent, they are stored and retrieved in isolation. Definitions, assumptions, and conditions established in the introduction or background sections of a document are invisible to a chunk retrieved from a later section. This *context decay* is especially damaging in legal and scientific texts, where a clause in section 8 may be legally conditioned on a definition given in section 2, or where an experimental result in section 4 is only interpretable in light of the methodology described in section 3. No existing chunking framework provides a mechanism to carry forward this accumulated contextual state into each retrieved unit.

Limitation 3: Failure of Single-Stage Retrieval for Multi-Hop Reasoning. Many practical queries over structured documents require multi-hop reasoning: the answer cannot be found in any single passage but must be assembled from evidence distributed across two or more weakly linked sections. Single-stage dense retrieval, which ranks passages by cosine similarity to the original query, retrieves only the most superficially similar chunk and misses the bridging evidence required to complete the reasoning chain. While iterative and graph-based approaches such as GraphRAG and multi-agent retrieval systems partially address this, they do so at the cost of constructing and maintaining expensive knowledge graphs or issuing multiple sequential LLM calls, making them impractical for latency-sensitive production environments.

2 Literature Review

The design of an effective RAG system depends critically on two components: how documents are segmented into retrievable units (chunking), and how relevant units are identified at query time (retrieval). This review surveys the key chunking strategies proposed in the literature, examines existing retrieval architectures, and identifies the gaps that motivate the proposed DR-RAG framework.

2.1 Chunking Strategies in RAG Systems

Chunking is the process of partitioning a source document into a collection of retrievable units. The granularity, coherence, and contextual completeness of these units directly determine the upper bound on retrieval quality. Table 1 summarises the principal chunking methods, their mechanisms, and their trade-offs.

2.1.1 Fixed-Size and Sentence-Based Chunking

Fixed-size chunking [3] is the most widely adopted approach due to its simplicity and deterministic behaviour. A sliding window of a fixed number of tokens (typically 256–512) is moved across the document, and each window becomes one chunk. While fast and easy to implement, this method is agnostic to sentence, clause, and paragraph boundaries. Legal clauses or theorem proofs that span more than one window are split mid-way, causing retrieval to return semantically incomplete chunks. Overlapping windows partially mitigate this but inflate index size and introduce redundancy.

Sentence-based chunking improves local coherence by using NLP tools (e.g., spaCy, NLTK punkt tokenizer) to detect sentence boundaries. However, it treats each sentence in isolation and ignores the structural hierarchy of the document. A legal sub-clause spread across three sentences may be split across two chunks, losing the conditional relationship between them.

2.1.2 Recursive and Semantic Chunking

Recursive chunking applies a priority-ordered list of delimiters (e.g., `\n\n`, `\n`, full stops) and recursively splits segments that exceed a maximum token limit. While more structure-aware than fixed-size methods, the boundaries produced are still syntactic rather than semantic, and the recursion depth is a manually tuned hyperparameter.

Semantic chunking [4] represents a significant improvement. It embeds each sentence and computes the cosine similarity between adjacent segments. A chunk boundary is inserted wherever the similarity drops below a predefined threshold, indicating a topic shift. This approach produces topically coherent chunks but requires an embedding model pass over the entire document at ingestion time, making it expensive for large corpora. It also lacks any mechanism to carry forward contextual information from earlier chunks.

Table 1: Comparison of Existing Chunking Methods for RAG Systems

Method	Description	Pros	Cons
Fixed-Size	Splits document into windows of n tokens with optional overlap	Simple; fast; predictable index size	Breaks semantic structure; mid-sentence splits common
Sentence-Based	Uses punctuation or NLP sentence detectors to set boundaries	Improves local coherence over fixed-size	Ignores paragraph and section hierarchy
Recursive	Applies a hierarchy of delimiters (paragraph, sentence, word)	Flexible granularity; structure-aware	Boundaries remain arbitrary; no semantic validation
Semantic	Computes embedding similarity between adjacent segments; splits on high drift	Produces semantically coherent chunks	Computationally expensive; sensitive to embedding quality
Proposition-Based	Decomposes text into atomic, self-contained factual propositions	High retrieval precision; minimal noise per chunk	Very slow; requires strong generative model; costly at scale
Page-Level	Treats each document page as a single chunk	Preserves physical layout and visual structure	Chunks too large; poor retrieval precision; cross-page context lost
Hierarchical (RAPTOR)	Builds a tree of progressively summarised clusters via recursive abstraction	Supports multi-granularity retrieval	Complex indexing pipeline; high memory overhead; slow to build

2.1.3 Proposition-Based Chunking

Proposition-based chunking [5] decomposes text into atomic, self-contained factual statements. Each proposition is a minimal claim that can be assessed independently. This maximises retrieval precision since each chunk is informationally pure. However, the extraction requires a capable instruction-following language model, making this method approximately 10–20× slower than fixed-size chunking. It is impractical for real-time or large-scale indexing pipelines.

2.1.4 Page-Level and Hierarchical (RAPTOR) Chunking

Page-level chunking is used in document-image pipelines where the physical layout carries meaning (e.g., scanned PDFs). Each page is treated as a single chunk. While this preserves visual structure, pages are too coarse as retrieval units and frequently span multiple unrelated topics.

RAPTOR [6] constructs a tree of summaries by recursively clustering and summarising document segments bottom-up. At query time, retrieval can occur at multiple levels of the tree, enabling both fine-grained and abstract matching. Despite its theoretical elegance, RAPTOR requires building and maintaining a complex tree index, incurs high memory overhead, and its summarisation quality degrades for highly technical documents such as legal contracts.

2.2 Retrieval Architectures

2.2.1 Single-Stage Dense Retrieval

Dense Passage Retrieval (DPR) [7] introduced bi-encoder models that independently encode queries and passages into a shared embedding space, enabling efficient maximum inner-product search (MIPS). DPR achieves strong performance on factoid open-domain QA benchmarks but is limited to single-hop questions where the answer is contained within a single passage. For multi-hop questions requiring evidence from multiple documents, its recall degrades substantially.

2.2.2 Multi-Stage and Iterative Retrieval

To address multi-hop reasoning, several iterative retrieval approaches have been proposed. RE-PLUG [8] treats the retrieval corpus as a plug-in module for black-box LLMs and uses ensemble scoring over multiple retrieved documents. Atlas [9] jointly fine-tunes a retriever and a reader using few-shot supervision, achieving strong results on knowledge-intensive tasks. However, both methods require repeated forward passes through large models and scale poorly with corpus size.

2.2.3 Graph-Based Retrieval

GraphRAG [10] constructs a knowledge graph from the document corpus and traverses entity-relation edges to answer complex queries. While it excels at reasoning over interconnected entities, graph construction is expensive and requires structured entity extraction pipelines. The latency of graph traversal also makes it unsuitable for real-time applications.

2.3 Research Gaps

A systematic analysis of the above literature reveals four critical gaps that existing methods fail to address simultaneously:

- 1. Absence of Document-Type-Aware Chunking.** All surveyed chunking methods treat documents uniformly, applying the same splitting strategy regardless of whether the input is a legal contract, a scientific paper, or a news article. Legal documents are structured around hierarchical clause trees, while academic papers follow IMRaD conventions. A chunking strategy that is agnostic to these structural patterns will systematically fragment semantically complete units.
- 2. Context Decay in Long Documents.** Fixed-size, sentence-based, and even semantic chunking methods discard all preceding content when constructing each chunk. In long documents, this leads to *context decay*: later chunks lose access to definitions, assumptions, or conditions established in earlier sections. No existing chunking framework incorporates a rolling contextual memory mechanism to counteract this decay.
- 3. Expensive Multi-Hop Retrieval.** Methods capable of multi-hop reasoning (RAPTOR, GraphRAG, iterative retrieval) impose high computational and latency costs. GraphRAG requires full graph construction before any query can be answered. Iterative retrievers require multiple sequential LLM calls. These costs are prohibitive for production deployments requiring sub-second response times.
- 4. Absence of Formal Chunking Quality Metrics.** The chunking literature lacks a formal, quantitative metric for evaluating whether a proposed boundary is semantically appropriate. Existing methods rely on downstream task performance (e.g., QA accuracy) as a proxy, making it difficult to diagnose chunking failures independently of retrieval and generation. A dedicated metric such as *semantic velocity* — measuring embedding drift at chunk boundaries — would enable principled, task-agnostic evaluation of chunking quality.

The proposed DR-RAG framework directly targets all four gaps through (i) SLM-guided, structure-aware boundary detection; (ii) rolling summary augmentation for context preservation; (iii) a lightweight classifier-routed two-stage retrieval pipeline; and (iv) semantic velocity as a formal chunking quality metric. The relationship between each gap and its corresponding system component is summarised in Table 2.

Table 2: Mapping of Research Gaps to Proposed System Components

#	Research Gap	DR-RAG Component
G1	No document-type-aware chunking	SLM-guided boundary detector with structure prompts (CAAC §1.2)
G2	Context decay in long documents	Rolling summary augmentation with velocity-regulated budget (CAAC §1.4)
G3	Expensive multi-hop retrieval	Classifier-routed two-stage retrieval; no graph required (DR-RAG §2.2–2.4)
G4	No formal chunking quality metric	Semantic velocity: embedding drift between adjacent chunks (CAAC §1.3)

3 Methodology

3.1 Objective

The primary objective of this work is to design an efficient and scalable Retrieval-Augmented Generation (RAG) framework that addresses two fundamental limitations of existing systems: (i) preservation of semantic document structure through adaptive, context-aware chunking, and (ii) accurate multi-hop reasoning via a lightweight classifier-routed two-stage retrieval mechanism.

The overall system is referred to as **DR-RAG** (Dynamic-Relevant RAG) and consists of two sequential phases: Phase I handles document ingestion and chunk construction, while Phase II handles query processing and response generation. The end-to-end pipeline takes an input document corpus and a user query and produces a generated response.

3.2 Phase I: Context-Augmented Agentic Chunking (CAAC)

3.2.1 Overview

Standard chunking methods partition a document into fixed-size token windows without regard for semantic or structural boundaries, causing context fragmentation in long structured documents such as legal contracts and research papers. Phase I addresses this by introducing **Context-Augmented Agentic Chunking (CAAC)**: a semantics-driven strategy guided by a Small Language Model (SLM) and regulated by a novel metric called *semantic velocity*.

3.2.2 Two-Pass Document Segmentation

To resolve the circular dependency between chunk boundaries and semantic velocity (which requires chunk embeddings that in turn depend on chunk boundaries), DR-RAG employs a **two-pass hierarchical segmentation process**.

Pass 1: Preliminary Sentence Embedding. The document is first segmented into individual sentences. Each sentence is embedded independently using the embedding model, and the distance between adjacent sentence embeddings is computed. These sentence-level drift values are used to generate preliminary chunk boundaries via the SLM boundary detector.

Pass 2: Adaptive Chunk Refinement. The preliminary chunks are then merged or split dynamically based on their semantic drift values:

- If drift is **low** (below a lower threshold), adjacent chunks are **merged** since they cover the same topic.
- If drift is **moderate** (between thresholds), the chunk boundary is **retained** as-is.
- If drift is **high** (above an upper threshold), the chunk is further **split** to isolate the topic change.

This two-pass design eliminates the circular dependency because sentence-level embeddings are computed *before* adaptive chunk boundaries are formed. Each final chunk corresponds to a logically complete unit of information (e.g., a legal clause, a theorem, or a methodology paragraph).

3.2.3 Boundary Detector Training Data

The SLM boundary detector is fine-tuned using a semi-supervised dataset constructed from:

- legal contracts (CUAD [2]),
- academic papers (arXiv / PubMed),
- manually annotated semantic transitions.

Weak supervision labels [11] are generated using heading transitions, paragraph topic shifts, and citation density changes. Human verification is applied to a sampled subset to ensure label quality.

3.2.4 Semantic Velocity

Semantic velocity quantifies the magnitude of meaning shift between two consecutive chunks. It is computed as the Euclidean distance between the embedding vectors of adjacent chunks — a higher value indicates a larger topic shift. Semantic velocity drives two dynamic control functions:

1. **Adaptive chunk sizing:** Low semantic velocity causes adjacent segments to be merged into a larger coherent chunk; high velocity enforces an immediate hard boundary.
2. **Summary length regulation:** Higher semantic velocity triggers a longer rolling summary to bridge the contextual gap introduced by the topic shift.

3.2.5 Context-Augmented Chunk Construction

Each raw chunk is augmented with a rolling summary generated from the **previous three chunks**. A window size of three was selected because a single preceding chunk provides insufficient long-range context, while more than five introduces excessive redundancy and token overhead; empirical pilot observations confirmed that three provides a practical balance between coherence and efficiency. The rolling summary is prepended (concatenated) to the original chunk content, producing the final *augmented chunk*.

3.2.6 Token Budget Scaling

The rolling-summary token budget scales linearly with semantic velocity. The budget ranges from a minimum of 32 tokens (for low drift) to a maximum of 128 tokens (for high drift). This means smooth topic continuity receives short summaries, while large topic shifts receive longer summaries to bridge the contextual gap.

3.2.7 Context Retention Score

To evaluate whether a chunk remains semantically understandable in isolation, we define the **Context Retention Score (CRS)**. CRS is calculated as the ratio of the cosine similarity between the chunk and its original surrounding document context [12] to the chunk’s token length. A higher CRS indicates that the chunk preserves more of its original meaning without depending on external context.

3.2.8 Embedding and Indexing

Each augmented chunk is encoded into a dense vector using the BGE-M3 [13] bi-encoder embedding model. All vectors are stored in a vector database (FAISS [14] / Qdrant [15]) and indexed with Hierarchical Navigable Small World (HNSW) graphs [16] for efficient approximate nearest-neighbour search at query time.

3.3 Phase II: Dynamic-Relevant RAG (DR-RAG)

3.3.1 Overview

Phase II defines the query-time pipeline. A lightweight classifier first determines the structural complexity of the incoming query. Simple factoid queries are answered with single-stage anchor retrieval. Complex multi-hop queries additionally trigger an evidence-mining stage, where intermediate facts extracted from anchor chunks are used to formulate a secondary query. The final answer is generated conditioned on the union of both retrieval sets.

3.3.2 Step 1 — Query Encoding and Routing

The incoming natural language query is encoded into a dense vector and passed to a **query routing classifier**. The classifier is implemented as a lightweight transformer-based binary classifier fine-tuned on labeled query-complexity data. It uses the following features:

- the query’s embedding vector,
- dependency-tree depth (syntactic complexity),
- conjunction frequency,
- semantic entropy score.

Based on these features, the classifier routes each query as either **single-hop** (answerable from one passage) or **multi-hop** (requires evidence from multiple passages).

Training Data. The classifier is trained using HotpotQA [1] multi-hop labels, Natural Questions [17] single-hop queries, and manually labeled legal QA pairs from CUAD [2].

Confidence Fallback. If the classifier confidence falls in the ambiguous range (between 45% and 55%), the system defaults to two-stage retrieval to maximise recall.

3.3.3 Step 2 — Anchor Retrieval

For all query types, the top- k most semantically similar chunks are retrieved from the vector index using cosine similarity, forming the *anchor set*. For single-hop queries, the anchor set serves directly as the final context. For multi-hop queries, it provides the seed for evidence mining.

3.3.4 Step 3 — Evidence Mining (Multi-Hop Only)

(a) Intermediate Fact Extraction. The fact extraction step uses **SLM-guided structured extraction**. Each anchor chunk is transformed into atomic propositions using prompt-based information extraction. The extraction process combines dependency parsing, Open Information Extraction (OpenIE) [18], and SLM prompt refinement. For example, given the chunk “*Clause 8 applies only when prior written consent is obtained,*” the extracted proposition would be the triple: (Clause 8, requires, WrittenConsent). This hybrid design reduces hallucinated facts while preserving semantic relationships.

(b) Mining Query Synthesis. The extracted facts are combined with the original query to synthesise an expanded secondary query that targets the missing evidence.

(c) Evidence Retrieval. The expanded query is encoded and used to retrieve additional evidence chunks from the vector index, forming the *evidence set*. Retrieved chunks that overlap with the anchor set are excluded to ensure complementary coverage.

3.3.5 Context Assembly and Response Generation

Anchor and evidence chunks are deduplicated using cosine similarity with a threshold of 0.95: if two chunks have a similarity score at or above this value, they are considered duplicates and the one with the higher retrieval relevance score is retained.

The language model then generates the final response autoregressively, conditioned on the query and the assembled context. The prompt passed to the generation model follows the structure: *[System Instruction] + [Assembled Context] + [Query]*.

3.4 Ablation Study

To isolate the contribution of each DR-RAG component, four ablation configurations will be evaluated:

Table 3: Ablation Study Configurations

Config	Removed Component
A1	No rolling summaries
A2	Fixed-size chunking instead of CAAC
A3	No query routing (all queries use two-stage)
A4	No evidence mining stage

Metrics compared across all configurations:

- Recall@K
- Exact Match (EM)
- Hallucination Rate
- Latency
- Context Retention Score (CRS)

This ablation determines which architectural component contributes most significantly to performance.

References

- [1] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 2369–2380.
- [2] D. Hendrycks, C. Burns, A. Chen, and S. Ball, “CUAD: An expert-annotated NLP dataset for legal contract understanding,” in *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2024.
- [5] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “Dense X retrieval: What retrieval granularity should we use?” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.
- [6] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, “RAPTOR: Recursive abstractive processing for tree-organized retrieval,” *arXiv preprint arXiv:2401.18059*, 2024.
- [7] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.
- [8] W. Yu, D. Iter, S. Wang, Y. Xu, M. Ju, S. Sanyal, C. Zhu, M. Zeng, and M. Jiang, “REPLUG: Retrieval-augmented black-box language models,” *arXiv preprint arXiv:2301.12652*, 2023.
- [9] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, “Atlas: Few-shot learning with retrieval augmented language models,” *Journal of Machine Learning Research (JMLR)*, vol. 24, no. 251, pp. 1–43, 2023.
- [10] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “GraphRAG: Unlocking LLM discovery on narrative private data,” Microsoft Research, Tech. Rep., 2024.

- [11] A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré, “Snorkel: Rapid training data creation with weak supervision,” in *Proceedings of the 44th International Conference on Very Large Data Bases (VLDB)*, vol. 11, no. 3, 2018, pp. 269–282.
- [12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.
- [13] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “BGE M3-Embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” *arXiv preprint arXiv:2402.03216*, 2024.
- [14] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [15] Qdrant Team, “Qdrant: High-performance vector search engine,” <https://qdrant.tech>, 2023.
- [16] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [17] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Chen, E. Jacob, I. Polosukhin, M. Kelcey, J. Devlin, K. Lee, K. Toutanova, L. Jones, M.-W. Chang, A. Dai, J. Uszkoreit, Q. Le, and S. Petrov, “Natural questions: A benchmark for question answering research,” *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 453–466, 2019.
- [18] G. Stanovsky, J. Michael, L. Zettlemoyer, and I. Dagan, “Supervised open information extraction,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2018, pp. 885–895.